Implement Client Script & UI Police (Incident)

❖ Project Planning Phase Overview

When implementing dynamic form behaviours on the Incident form, your roadmap should follow these core phases: requirement analysis, design strategy (leveraging low-code features first), development, testing, and deployment.

Phase 1: Requirements Gathering & Scope Definition

Identify Business Rules: Document exact triggers (e.g., “When Incident State changes to Resolved, make Resolution Code and Resolution Notes mandatory”).

Determine Target Views: Decide if the rule applies globally or to specific view types (e.g., Workspace vs. Core UI).

Data Security vs. Presentation: Remember that UI Policies and Client Scripts are client-side controls. Always back them up with Data Policies or Business Rules if strict backend data enforcement is needed.

Phase 2: Design & Best Practice Strategy

ServiceNow best practices dictate a specific hierarchy for form modifications to prevent race conditions and performance degradation.

Prioritize UI Policies: Use native UI Policies instead of Client Scripts whenever possible to handle simple field visibility, read-only status, and mandatory attributes. They load faster and require no code maintenance.

Reserve Client Scripts for Complexity: Use Client Scripts (onChange, onLoad, onSubmit) only when you need advanced logic, DOM manipulation, old-value comparisons, or asynchronous server calls via GlideAjax.

Avoid Logic Overlaps: Never control the exact same field attribute via a Client Script and a UI Policy simultaneously, as conflicting execution orders lead to unpredictable form behavior.

Phase 3: Step-by-Step Implementation Plan

Step A: Build UI Policies (Low-Code Layer)

Navigate to System UI > UI Policies (or use Studio).

Target Table: Incident (incident).

Define conditions clearly (e.g., State [is] On Hold).

Configure UI Policy Actions to dictate whether target fields become Mandatory, Visible, or Read-Only.

Step B: Build Client Scripts (Scripting Layer)

Navigate to System Definition > Client Scripts.

Target Table: Incident (incident).

Ensure proper use of parameters (e.g., checking isLoading in onChange scripts to avoid unexpected onload triggers).

Phase 4: Testing & Quality Assurance

Cross-Browser & Workspace Testing: Validate that the UI Policy or Client Script behaves identically in both the standard Core UI and the Next Experience / Agent Workspace.

Execution Order Validation: Confirm that client-side logic executes smoothly without lagging form load times.

Data Integrity Check: Verify that mandatory fields enforced on the form are also safeguarded at the database level if required for compliance.